SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1708**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

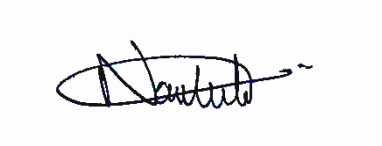
Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **NANDHAKUMAARAN D(192511312)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1 – Hospital Shift Assignment (Backtracking Search)

Problem formulation (CSP)

- **Variables:** D1, D2, D3 (one per doctor)
- **Domain:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N
- **Constraints:**
  - C1: D1 ≠ Night
  - C2: shift(D2) < shift(D3)
  - C3: all different (only one doctor per shift, and each doctor gets exactly one shift)
  - C4: D3 ≠ Morning

Variables are assigned in the order D1 → D2 → D3, and values are tried in the order M, A, N. After every assignment the constraints that involve only assigned variables are checked; on a violation the algorithm backtracks.

Step-by-step search trace

Step	Partial assignment	Constraint check	Result
1	D1 = Morning	C1 ✓ (not Night); C3 ✓	Accept
2	D1 = M, D2 = Morning	C3 ✗ (Morning already used by D1)	Reject → next value
3	D1 = M, D2 = Afternoon	C3 ✓	Accept
4	D1 = M, D2 = A, D3 = Morning	C4 ✗ (D3 cannot work Morning); C3 ✗	Reject → next value
5	D1 = M, D2 = A, D3 = Afternoon	C3 ✗ (Afternoon used by D2); C2 ✗ (A is not before A)	Reject → next value

6	D1 = M, D2 = A, D3 = Night	C1 ✓; C2 ✓ (A < N); C3 ✓; C4 ✓; all doctors assigned	SOLUTION FOUND

Continuing the search to see the remaining branches (optional check):

Step	Partial assignment	Constraint check	Result
7	D1 = M, D2 = Night	C3 ✓	Accept
8	D2 = N, D3 = Morning / Afternoon / Night	Morning ✗ (C4); Afternoon ✗ (C2: N is not before A); Night ✗ (C3)	Backtrack to D1
9	D1 = Afternoon, D2 = Morning	C1 ✓; C3 ✓	Accept
10	D1 = A, D2 = M, D3 = Night	Morning ✗ (C4), Afternoon ✗ (C3), Night ✓ (C2: M < N)	2nd solution
11	D1 = A, D2 = Night, D3 = any	Morning ✗ (C4), Afternoon ✗ (C3), Night ✗ (C3)	Backtrack
12	D1 = Night	C1 ✗ (D1 cannot work Night)	Pruned

Final valid schedule (first solution found by backtracking):

Doctor	Shift	Constraints
D1	Morning	C1 ✓
D2	Afternoon	C2 ✓
D3	Night	C2, C4 ✓

Note: (D1 = Afternoon, D2 = Morning, D3 = Night) is also valid, so the problem has exactly two solutions. Backtracking returns the first one it meets.

Question 2 – Robot in a 5×5 Grid (BFS and Manhattan-Distance Priority Queue)

Assumptions

The question does not include the grid drawing, so the following 5×5 grid is assumed (coordinates are (row, column), starting from (0,0) at the top-left). Replace it with your own grid if your sheet gives a different layout; the method stays the same.

S			**X**	
	X		**X**	
	X			
			X	
				G

S = Start (0,0) · G = Goal (4,4) · X = obstacles at (0,3), (1,1), (1,3), (2,1), (3,3)

- Moves: Up, Down, Left, Right (expanded in this order), cost of each move = 1; obstacles cannot be entered.
- Manhattan heuristic: $h(n) = |row - 4| + |col - 4|$, so $h(S) = 8$. It is admissible and consistent on a 4-connected grid with unit costs.
- The question mentions both BFS and a priority queue with Manhattan distance. BFS uses a plain FIFO queue, while a priority queue with Manhattan distance is A* ($f = g + h$). Both are shown below.

Part A – BFS (FIFO queue)

Algorithm: put S in the queue; repeatedly remove the front node, generate its unvisited nonobstacle neighbours (Up, Down, Left, Right) and append them to the back; stop when G is removed.

Step	Node expanded	New nodes added	Queue after the step
1	(0,0)	(1,0), (0,1)	(1,0), (0,1)
2	(1,0)	(2,0)	(0,1), (2,0)
3	(0,1)	(0,2)	(2,0), (0,2)
4	(2,0)	(3,0)	(0,2), (3,0)
5	(0,2)	(1,2)	(3,0), (1,2)
6	(3,0)	(4,0), (3,1)	(1,2), (4,0), (3,1)
7	(1,2)	(2,2)	(4,0), (3,1), (2,2)
8	(4,0)	(4,1)	(3,1), (2,2), (4,1)
9	(3,1)	(3,2)	(2,2), (4,1), (3,2)
10	(2,2)	(2,3)	(4,1), (3,2), (2,3)
11	(4,1)	(4,2)	(3,2), (2,3), (4,2)

12	(3,2)	none	(2,3), (4,2)
13	(2,3)	(2,4)	(4,2), (2,4)
14	(4,2)	(4,3)	(2,4), (4,3)
15	(2,4)	(1,4), (3,4)	(4,3), (1,4), (3,4)
16	(4,3)	(4,4)	(1,4), (3,4), (4,4)
17	(1,4)	(0,4)	(3,4), (4,4), (0,4)
Step	**Node expanded**	**New nodes added**	**Queue after the step**
18	(3,4)	none	(4,4), (0,4)
19	(4,4)	Goal reached	(0,4)

BFS path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Cost = 8 moves. 19 nodes were expanded. BFS is optimal here because every move costs 1.

Part B – Priority queue with Manhattan distance (A*)

Algorithm: the priority queue is ordered by f(n) = g(n) + h(n), where g is the path cost so far. Ties are broken by smaller g, then by row-major order. Nodes already expanded are not reinserted.

Node expanded	g	h	f	Nodes pushed (with f)	Priority queue after the step (node:f)
(0,0)	0	8	8	(1,0) f=8; (0,1) f=8	(0,1):8; (1,0):8
(0,1)	1	7	8	(0,2) f=8	(1,0):8; (0,2):8
(1,0)	1	7	8	(2,0) f=8	(0,2):8; (2,0):8
(0,2)	2	6	8	(1,2) f=8	(2,0):8; (1,2):8
(2,0)	2	6	8	(3,0) f=8	(1,2):8; (3,0):8
(1,2)	3	5	8	(2,2) f=8	(3,0):8; (2,2):8
(3,0)	3	5	8	(4,0) f=8; (3,1) f=8	(2,2):8; (3,1):8; (4,0):8
(2,2)	4	4	8	(3,2) f=8; (2,3) f=8	(3,1):8; (4,0):8; (2,3):8; (3,2):8
(3,1)	4	4	8	(4,1) f=8	(4,0):8; (2,3):8; (3,2):8; (4,1):8
(4,0)	4	4	8	none	(2,3):8; (3,2):8; (4,1):8
(2,3)	5	3	8	(2,4) f=8	(3,2):8; (4,1):8; (2,4):8
(3,2)	5	3	8	(4,2) f=8	(4,1):8; (2,4):8; (4,2):8
(4,1)	5	3	8	none	(2,4):8; (4,2):8

(2,4)	6	2	8	(1,4) f=10; (3,4) f=8	(4,2):8; (3,4):8; (1,4):10
(4,2)	6	2	8	(4,3) f=8	(3,4):8; (4,3):8; (1,4):10
(3,4)	7	1	8	(4,4) f=8	(4,3):8; (4,4):8; (1,4):10
(4,3)	7	1	8	none	(4,4):8; (1,4):10
(4,4)	8	0	8	Goal reached	-

A* path: (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4)
Cost = 8 moves. The optimal cost is 8, which equals the Manhattan distance from S to G, so no detour is needed. The BFS path and the A* path differ only because several shortest paths exist; both cost 8.

Optimal path and cost: 8 (equal to the lower bound h(S) = 8, hence optimal).

Question 3 – Autonomous Rescue Robot (Partially Observable, Online Search)

a) Analysis of the constraints and their effect on decision-making

Constraint	Effect on decision-making
Moves: up, down, left, right	Fixes the action set to 4 actions and the state space to a grid of cells. Branching factor is at most 4, and moves are reversible, so the robot can back out of dead ends.
Some cells are obstacles	Actions into blocked cells are illegal. The robot must discover them and replan, and it cannot assume a straight path to G.
Limited sensing (adjacent cells only)	The environment is partially observable. The robot sees only a 1-cell neighbourhood, so it cannot compute a complete plan in advance and must keep a belief/map of the world. Decisions are made with incomplete information.
Move cost = 1; entering a risky zone costs +2 extra (total 3)	The problem is a weighted-cost problem, so plain BFS is no longer optimal. A cost-aware algorithm (UCS / A*) is needed, and the robot may prefer a longer but safer route: a detour of up to 2 extra steps is cheaper than one risky cell.
Minimise total cost while staying safe	The objective is optimality plus safety. Risky cells are avoided unless they save more than 2 units of cost, and the robot should never step into a cell it knows is blocked or dangerous.

Update knowledge dynamically (online search)	Planning and acting are interleaved. After each move the robot senses neighbours, updates its map and cost estimates, and re-plans if the current plan is invalid. This makes the search online, not offline.

Overall: because of partial observability and unknown obstacles, the robot cannot solve the task by offline planning. It must repeatedly sense, update its model, choose the cheapest safe next step, and act.

b) Solution approach: Online Uniform Cost Search with re-planning

Chosen strategy: Uniform Cost Search (UCS) run on the robot's current knowledge map and repeated after every new observation (online search with re-planning; the same idea as D* Lite / LRTA*). Steps:

1. Initialise: the knowledge map contains only S and G. All other cells are marked unknown, and unknown cells are treated as free with cost 1 (free-space assumption). Set the current position = S.
2. Sense: observe the four adjacent cells and record each as free, obstacle, or risky. Update the map.
3. Plan: run UCS from the current cell to G on the known map. Each node is expanded in order of the lowest path cost g(n), where the step cost is 1 for a normal cell and 1 + 2 = 3 for a risky cell. Obstacles are removed from the graph.
4. Act: execute only the first move of the cheapest plan found, then update the current position and the total cost so far.
5. Check and re-plan: if the newly sensed cells block or make the current path more expensive, discard the old plan and repeat from step 2. Otherwise keep following the plan.
6. Terminate: stop when the current cell is G (survivor found). If the priority queue becomes empty without reaching G, no safe path exists with the knowledge gathered so far; the robot then explores unknown frontier cells or reports failure.
7. Memory: keep the visited cells and the map so the robot does not repeat dead ends; this also guarantees the search terminates in a finite grid.

Priority-queue behaviour: the frontier is ordered by g(n). A cell with risk has g increased by 3 instead of 1, so it is expanded later than a safe cell at the same distance. The robot therefore prefers safe corridors, and enters a risky zone only when every safe alternative costs more than the extra 2 units.

Optional improvement: adding the Manhattan distance as a heuristic (A*) reduces the number of expanded nodes while keeping optimality, because h never overestimates when the minimum step cost is 1.

c) Application of AI concepts

Intelligent agent. The robot is a goal-based, utility-based agent that follows the sense → think → act loop. Its PEAS description is:

PEAS	Description
Performance measure	Reach the survivor at G, with minimum total path cost and no unsafe moves
Environment	Disaster-affected building modelled as a grid of rooms with obstacles and risky zones
Actuators	Movement motors (up, down, left, right)
Sensors	Short-range sensors that observe the adjacent cells only

Rationality. A rational agent chooses the action that maximises its expected performance measure given its percept history and built-in knowledge. Here, the robot picks the next move on the cheapest safe plan computed from its current map, avoids known obstacles and risky cells unless they are worth the cost, and re-plans when new percepts arrive. It is rational relative to what it knows, not omniscient: it can still make an unlucky choice if a hidden obstacle appears, but it corrects it as soon as it senses it.

Environment type.

Property	Type	Reason
Observability	Partially observable	Only adjacent cells can be sensed
Agents	Single agent	Only the robot acts (survivors are passive)
Determinism	Deterministic (given the known cell)	A move into a free cell always succeeds; uncertainty comes from missing knowledge, not from random outcomes
Episodic / Sequential	Sequential	Each move affects later states and the total cost
Static / Dynamic	Static (mostly)	The layout does not change while the robot plans, though new knowledge is revealed over time
Discrete / Continuous	Discrete	Grid cells and four discrete moves
Known / Unknown	Unknown	The map is not known in advance and has to be learned by exploration

d) Why this approach is the most suitable

- Handles costs: UCS accounts for the +2 risk penalty, unlike BFS, which treats all steps as equal and could lead the robot through risky cells.
- Optimal and complete: on the known map, UCS always returns the minimum-cost path and finds one if it exists (all step costs are positive).
- Fits partial observability: re-planning after every observation lets the robot work with only adjacent-cell sensing, which an offline planner cannot do.
- Supports safety: risky cells are expensive, so the plan avoids them naturally without a separate rule.
- Adapts dynamically: new obstacles discovered at run time are added to the map and the cheapest path is recomputed immediately.
- Simple and efficient on a small grid: the cost of repeated UCS is low. If the grid is large, the same framework can be upgraded to A* / D* Lite with the Manhattan heuristic.
- Alternatives are weaker: DFS is neither optimal nor cost-aware, greedy best-first can walk into expensive or risky regions, and a plain offline A* assumes full knowledge of the map.

Conclusion: online Uniform Cost Search with dynamic map updates gives the lowest total cost while keeping the rescue robot safe in a partially observable building.